

Documentação Vyra

Gabrielle Calazans RM 564460

João Felipe Bertini RM 563478

Pedro Batista RM 563220

VYRA — Ecossistema de Carreiras Vivas

(CLI + API)

Plataforma educacional que simula um **ecossistema de carreiras** “vivas”, reagindo a tendências (IA, clima, trabalho remoto) e recomendando **trilhas profissionais** com base no **DNA profissional digital** do usuário.

- **CLI**: menu interativo com relatórios, gráficos e export de CSV.
- **API (FastAPI)**: endpoints para integrar com front-end/web (Swagger incluso).

Slogan: “Carreiras que nascem, crescem e se transformam — como você.”

Sumário

- Arquitetura
- Estrutura de Pastas
- Instalação
- Como Rodar (CLI)
- Como Rodar (API)
- Fluxo de Uso (Passo a Passo)
- Conceitos e Modelagem
- Cálculo de Score e Explainability
- Persistência (JSON/CSV/PNG)
- Relatórios e Gráficos
- Documentação da API
- Requisitos da Disciplina (Checklist)
- Boas Práticas (Git / .gitignore)
- Solução de Problemas

Arquitetura

[Usuário]

- └ CLI (terminal) → prints/CSV/PNGs
- └ Front (opcional) → chama API
 - └ FastAPI (Swagger) → reusa o core

[Core VYRA (Python)]

- └ careers.py (DF de carreiras + atualização por tendências)
- └ world.py (entrada de tendências)
- └ users.py (entrada do DNA)
- └ recommend.py (score + rationale)
- └ reports.py (tabelas pandas long/wide + export CSV)
- └ analytics.py (contagens e gráficos matplotlib)
- └ storage.py (persistência JSON/CSV/PNGs)
- └ io.py (apresentação/prints no CLI)

Estrutura de Pastas

Vyra-Core-Python/

- └ main.py
- └ requirements.txt
- └ data/ # gerado em runtime (JSON/CSV/PNGs)
 - └ users.json
 - └ careers.json
 - └ trends.json
 - └ reports/
 - └ recomendacoes_long.csv
 - └ recomendacoes_wide.csv
 - └ estado_counts.png
 - └ ods_por_estado.png
- └ vyra/
 - └ __init__.py
 - └ io.py
 - └ careers.py
 - └ world.py
 - └ users.py
 - └ recommend.py
 - └ reports.py
 - └ analytics.py
 - └ storage.py

```
└─ api/  
  └─ app.py          # FastAPI (Swagger em /docs)
```

Instalação

Recomendado: **Python 3.11 (x64) + venv**.

```
# Windows (PowerShell)  
py -3.11-64 -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip setuptools wheel  
pip install -r requirements.txt
```

Teste rápido:

```
python -c "import pandas, matplotlib, fastapi, uvicorn; print('ok')"
```

Como Rodar (CLI)

```
python .\main.py
```

Menu principal (exemplo):

- [1] Listar carreiras
- [2] Atualizar ecossistema (IA/Clima/Remoto)
- [3] Ver contagem por estado evolutivo
- [4] Cadastrar DNA profissional (usuário)
- [5] Recomendar para o último DNA cadastrado
- [6] Listar DNAs cadastrados
- [7] Salvar DNAs (JSON)
- [8] Carregar DNAs (JSON)
- [9] Relatório pandas (users × recomendações)
- [10] Salvar ecossistema (carreiras + estado)
- [11] Carregar ecossistema salvo
- [12] Analytics & Gráficos

Como Rodar (API)

```
python -m uvicorn api.app:app --reload  
# Swagger: http://127.0.0.1:8000/docs  
# Health: http://127.0.0.1:8000/health
```

A API carrega/salva automaticamente data/users.json, data/careers.json e data/trends.json.

Fluxo de Uso (Passo a Passo)

1. **Atualizar ecossistema** com tendências (CLI [2] ou API PUT /v1/trends).
2. **Cadastrar usuário (DNA)** (CLI [4] ou API POST /v1/users).
3. **Gerar recomendações** (CLI [5] ou API POST /v1/recommend).
4. **Relatórios**: tabela long/wide + CSV (CLI [9] ou API /v1/reports/recommendations).
5. **Analytics**: contagem por estado + ODS (CLI [12] → gráficos em data/reports).

Conceitos e Modelagem

- **Carreira (espécie)**: linha no DataFrame com carreira, skills_base, ods e sensibilidades impacto_ia, impacto_clima, impacto_remoto.
- **Tendências do mundo**: valores 0..10 para ia, clima, remoto.
- **Estado evolutivo**: calculado por score ponderado → mutar, crescer, nascer, em_risco, extinta.
- **DNA profissional**: nome, skills, valores, modo (remoto|hibrido|presencial), ods_interesse.

Cálculo de Score e Explainability

No `vyra/recommend.py`, para cada carreira:

- **Skills:** +2 por skill presente no DNA, -1 por gap.
- **ODS:** +2 por ODS em comum entre usuário e carreira.
- **Estado:** mutar +3, crescer +2, nascer +1, em_risco -1, extinta -3.
- **Modo de trabalho x impacto remoto:**
 - remoto & $\geq 7 \rightarrow +2$
 - presencial & $\leq 3 \rightarrow +2$
 - híbrido & 4..6 $\rightarrow +1$

A API pode retornar **rationale** (quando `explain=true`), detalhando o score:

```
"rationale": { "skills": 4, "gap": -1, "ods": 2, "estado": 2,
"modo": 2, "total": 9 }
```

Persistência (JSON/CSV/PNG)

- **users.json:** DNAs cadastrados.
- **careers.json:** ecossistema com estado_evolutivo.
- **trends.json:** tendências aplicadas.
- **CSV** (relatórios): data/reports/recomendacoes_long.csv e recomendacoes_wide.csv.
- **Gráficos** (PNG): data/reports/estado_counts.png, ods_por_estado.png.

storage.py centraliza salvar/carregar.

Relatórios e Gráficos

- **reports.py:**
 - `build_recommendations_table` $\rightarrow$ tabela **longa** (uma linha por recomendação).
 - `build_wide_from_long` $\rightarrow$ tabela **larga** (colunas `rec_1..rec_3`).
 - `export_csv` $\rightarrow$ CSV UTF-8.
- **analytics.py:**
 - `state_counts` $\rightarrow$ contagem por estado.
 - `ods_distribution_by_state` $\rightarrow$ ODS x estado (tabela).
 - `plot_*` $\rightarrow$ gráficos **matplotlib** (sem estilos explicitados).